



2. Introduction to Scikit-Learn and Model Evaluation

Overview

*After exploring the response variable of the case study data, this chapter introduces the core functionality of scikit-learn for training models and making predictions, through simple use cases of logistic and linear regression. Evaluation metrics for binary classification models, including **true and false positive rates**, the **confusion matrix**, the **receiver operating characteristic (ROC) curve**, and the **precision-recall curve**, are demonstrated both from scratch and using convenient scikit-learn functionality. By the end of this chapter, you'll be able to build and evaluate binary classification models using scikit-learn.*

Introduction

In the previous chapter, you became familiar with basic Python and then learned about the pandas tool for data exploration. Using Python and pandas, you performed operations such as loading a dataset, verifying data integrity, and performing exploratory analysis of the features, or independent variables, in the data.

In this chapter, we will finish our exploration of the data by examining the response variable. After we've concluded that the data is of high quality and makes sense, we will be ready to move forward with developing machine learning models. We will take our first steps with scikit-learn, one of the most popular machine learning packages avail-

able in the Python language. Before learning the details of how mathematical models work in the next chapter, here we'll start to get comfortable with the syntax for using them in scikit-learn.

We will also learn some common techniques for answering the question, "Is this model good or not?" There are many possible ways to approach model evaluation. For business applications, a financial analysis to determine the value that could be created by a model is an important way to understand the potential impact of your work. Usually, it's best to scope the business opportunity of a project at the very beginning. However, as the emphasis of this book is on machine learning and predictive modeling, we will demonstrate a financial analysis in the final chapter.

There are several important model evaluation criteria that are considered standard knowledge in data science and machine learning. We will cover a few of the most widely used classification model performance metrics here.

Exploring the Response Variable and Concluding the Initial Exploration

We have now looked through all the **features** to see whether any data is missing, as well as to generally examine them. The features are important because they constitute the **inputs** to our machine learning algorithm. On the other side of the model lies the **output**, which is a prediction of the **response variable**. For our problem, this is a binary flag indicating whether or not a credit account will default next month.

The key task for the case study project is to come up with a predictive model for this target. Since the response variable is a yes/no flag, this problem is called a **binary classification** task. In our labeled data, the samples (accounts) that defaulted (that is, '**default payment next**

`month' = 1)` are said to belong to the **positive class**, while those that didn't belong to the **negative class**.

The main piece of information to examine regarding the response of a binary classification problem is this: what is the proportion of the positive class? This is an easy check.

Before we perform this check, we load the packages we need with the following code:

```
import numpy as np #numerical computation
import pandas as pd #data wrangling
import matplotlib.pyplot as plt #plotting package
#Next line helps with rendering plots
%matplotlib inline
import matplotlib as mpl #add'l plotting functionality
mpl.rcParams['figure.dpi'] = 400 #high res figures
```

Now we load the cleaned version of the case study data like this:

```
df =
pd.read_csv('../Data/Chapter_1_cleaned_data.csv')
```

Note

The cleaned dataset should have been saved as a result of your work in Chapter 1, Data Exploration and Cleaning. The path to the cleaned data in the preceding code snippet may be different if you saved it in a different location.

Now, to find the proportion of the positive class, all we need to do is get the average of the response variable over the whole dataset. This has the interpretation of the default rate. It's also worthwhile to check the number of samples in each class, using **groupby** and **count** in pandas. This is presented in the following screenshot:

```
df['default payment next month'].mean()

0.2217971797179718

df.groupby('default payment next month')['ID'].count()

default payment next month
0      20750
1       5914
Name: ID, dtype: int64
```

Figure 2.1: Class balance of the response variable

Since the target variable is **1** or **0**, taking the mean of this column indicates the fraction of accounts that defaulted: 22%. The proportion of samples in the positive class (default = 1), also called the **class fraction** for this class, is an important statistic. In binary classification, datasets are described in terms of being **balanced** or **imbalanced**: are the proportions of the positive and negative classes equal or not? Most machine learning classification models are designed to work with balanced data: a 50/50 split between the classes.

However, in practice, real data is rarely balanced. Consequently, there are several methods geared toward dealing with imbalanced data. These include the following:

- **Undersampling** the majority class: Randomly throwing out samples from the majority class until the class fractions are equal, or at least less imbalanced.
- **Oversampling** the minority class: Randomly adding duplicate samples of the minority class to achieve the same goal.
- **Weighting samples**: This method is performed as part of the training step, so the minority class collectively has as much "emphasis" as the majority class in the trained model. The effect of this is similar to oversampling.
- More sophisticated methods, such as **Synthetic Minority Oversampling Technique (SMOTE)**.

While our data is not, strictly speaking, balanced, we also note that a positive class fraction of 22% is not particularly imbalanced, either.

Some domains, such as fraud detection, typically deal with much smaller positive class fractions: on the order of 1% or less. This is because the proportion of "bad actors" is quite small compared to the total population of transactions; at the same time, it is important to be able to identify them if possible. For problems like this, it is more likely that using a method to address class imbalance will lead to substantially better results.

Now that we've explored the response variable, we have concluded our initial data exploration. However, data exploration should be considered an ongoing task that you should continually have in mind during any project. As you create models and generate new results, it's always good to think about what those results imply about the data, which usually requires a quick iteration back to the exploration phase. A particularly helpful kind of exploration, which is also typically done before model building, is examining the relationship between features and the response. We gave a preview of that in *Chapter 1, Data Exploration and Cleaning*, when we were grouping by the **EDUCATION** feature and examining the mean of the response variable. We will also do more of this later. However, this has more to do with building a model than checking the inherent quality of the data.

The initial perusal through all the data that we have just completed is an important foundation to lay at the beginning of a project. As you do this, you should ask yourself the following questions:

- Is the data **complete**?
Are there missing values or other anomalies?
- Is the data **consistent**?
Does the distribution change over time, and if so, is this expected?
- Does the data **make sense**?
Do the values of the features fit with their definition in the data dictionary?

The latter two questions help you determine whether you think the data is **correct**. If the answer to any of these questions is "no," this should be addressed before continuing the project.

Also, if you think of any alternative or additional data that might be helpful to have and is possible to get, now would be a good point in the project life cycle to augment your dataset with it. Examples of this may include postal code-level demographic data, which you could **join** to your dataset if you had the addresses associated with accounts. We don't have these for the case study data and have decided to proceed on this project with the data we have now.

Introduction to Scikit-Learn

While pandas will save you a lot of time loading, examining, and cleaning data, the machine learning algorithms that will enable you to do predictive modeling are located in other packages. Scikit-learn is a foundational machine learning package for Python that contains many useful algorithms and has also influenced the design and syntax of other machine learning libraries in Python. For this reason, we focus on scikit-learn to develop skills in the practice of predictive modeling. While it's impossible for any one package to offer everything, scikit-learn comes pretty close in terms of accommodating a wide range of classic approaches for classification, regression, and supervised learning. However, it does not offer much functionality for some more recent advancements, such as deep learning.

Here are a few other related packages you should be aware of:

SciPy:

- Most of the packages we've used so far, such as NumPy and pandas, are actually part of the SciPy ecosystem.
- SciPy offers lightweight functions for classic methods such as linear regression and linear programming.

StatsModels:

- More oriented toward statistics and maybe more comfortable for users familiar with R
- Can get p-values and confidence intervals on regression coefficients
- Capability for time series models such as ARIMA

XGBoost and LightGBM:

- Offer a suite of state-of-the-art ensemble models that often outperform random forests. We will learn about XGBoost in *Chapter 6*, *Gradient Boosting*, *SHAP Values*, and *Dealing with Missing Data*.

TensorFlow, Keras, and PyTorch:

- Deep learning capabilities

There are many other Python packages that may come in handy, but this gives you an idea of what's out there.

Scikit-learn offers a wealth of different models for various tasks, but, conveniently, the syntax for using them is consistent. In this section, we will illustrate model syntax using a **logistic regression** model. Logistic regression, despite its name, is actually a classification model. This is one of the simplest, and therefore most important, classification models. In the next chapter, we will go through the mathematical details of how logistic regression works. Until then, you can simply think of it as a black box that can learn from labeled data, then make predictions.

From the first chapter, you should be familiar with the concept of training an algorithm on labeled data so that you can use this trained model to then make predictions on new data. Scikit-learn encapsulates these core functionalities in the **.fit** method for training models, and the **.predict** method for making predictions. Because of the consis-

tent syntax, you can call `.fit` and `.predict` on any scikit-learn model from linear regression to classification trees.

The first step is to choose some model, in this example a logistic regression model, and instantiate it from the **class** provided by scikit-learn. In Python, classes are templates for creating objects, which are collections of functions, like `.fit`, and data, such as information learned from the model fitting process. When you instantiate a model class from scikit-learn, you are taking the blueprint of the model that scikit-learn makes available to you and creating a useful **object** out of it. You can train this object on your data and then save it to disk for later use. The following snippets can be used to perform this task. The first step is to import the class:

```
from sklearn.linear_model import LogisticRegression
```

The code to instantiate the class into an object is as follows:

```
my_lr = LogisticRegression()
```

The object is now a variable in our workspace. We can examine it using the following code:

```
my_lr
```

This should give the following output:

```
LogisticRegression()
```

Notice that the act of creating the model object involves essentially no knowledge of what logistic regression is or how it works. Although we didn't select any particular options when creating the logistic regression model object, we are now in fact using many **default options** for how the model is formulated and would be trained. In effect, these are choices we have made regarding the details of model implementation without having been aware of it. The danger of an easy-to-use package such as scikit-learn is that it has the potential to obscure these choices from you. However, any time you use a machine learning model that has been prepared for you as scikit-learn models have been, your first

job is to understand all the options that are available. A best practice in such cases is to explicitly provide every keyword parameter to the model when you create the object. Even if you are just selecting all the default options, this will help increase your awareness of the choices that are being made.

We will review the interpretation of these choices later on, but for now here is the code for instantiating a logistic regression model with all the default options:

```
my_new_lr = LogisticRegression(penalty='l2',  
                                dual=False,\n\n                                tol=0.0001, C=1.0,\n                                fit_intercept=True,\n                                intercept_scaling=1,\n                                class_weight=None,\n                                random_state=None,\n                                solver='lbfgs',\n                                max_iter=100,\n                                multi_class='auto',\n                                verbose=0,\n\n                                warm_start=False,\n\n                                n_jobs=None,\n\n                                l1_ratio=None)
```

Even though the object we've created here in `my_new_lr` is identical to `my_lr`, being explicit like this is especially helpful when you are starting out and learning about different kinds of models. Once you're more comfortable, you may wish to just instantiate with the default options and make changes later as necessary. Here, we show how this may be done. The following code sets two options and displays the current state of the model object:

```
my_new_lr.C = 0.1  
my_new_lr.solver = 'liblinear'  
my_new_lr
```

This should produce the following:

```
Out[11]: LogisticRegression(C=0.1, solver='liblinear')
```

Notice that only the options we have updated from the default values are displayed. Here, we've taken what is called a **hyperparameter** of the model, **C**, and updated it from its default value of **1** to **0.1**. We've also specified a solver. For now, it is enough to understand that hyperparameters are options that you supply to the model, before fitting it to the data. These options specify the way in which the model will be trained. Later, we will explain in detail what all the options are and how you can effectively choose values for them.

To illustrate the core functionality, we will fit this nearly default logistic regression to some data. Supervised learning algorithms rely on labeled data. That means we need both the features, customarily contained in a variable called **x**, and the corresponding responses, in a variable called **y**. We will borrow the first 10 samples of one feature, and the response, from our dataset to illustrate:

```
X = df['EDUCATION'][0:10].values.reshape(-1,1)
X
```

That should show the values of the **EDUCATION** feature for the first 10 samples:

```
array([[2],
       [2],
       [2],
       [2],
       [2],
       [1],
       [1],
       [2],
       [3],
       [3]])
```

Figure 2.2: First 10 values of a feature

The corresponding first 10 values of the response variable can be obtained as follows:

```
y = df['default payment next month'][0:10].values  
y
```

Here is the output:

```
Out[13]: array([1, 1, 0, 0, 0, 0, 0, 0, 0, 0])
```

Here, we have selected a couple of Series (that is, columns) from our DataFrame: the **EDUCATION** feature we've been discussing, and the response variable. Then we selected the first 10 elements of each and finally used the **.values** method to return NumPy arrays. Also notice that we used the **.reshape** method to reshape the features. Scikit-learn expects that the first dimension (that is, the number of rows) of the array of features will be equal to the number of samples, so we need to make that reshaping for **x**, but not for **y**. The **-1** in the first positional argument of **.reshape** means to make the output array shape flexible in that dimension, according to how much data goes in. Since we just have a single feature in this example, we specified the number of columns as the second argument, **1**, and let the **-1** argument indicate that the array should "fill up" along the first dimension with as many elements as necessary to accommodate the data, in this case, 10 elements. Note that while we've extracted the data into NumPy arrays to show how this can be done, it's also possible to use pandas Series as direct input to scikit-learn.

Let's now use this data to fit our logistic regression. This is accomplished with just one line:

```
my_new_lr.fit(X, y)
```

Here is the output:

```
Out[14]: LogisticRegression(C=0.1, solver='liblinear')
```

That's all there is to it. Once the data is prepared and the model is specified, fitting the model almost seems like an afterthought. Of course, we are ignoring all the important options and what they mean right now. But, technically speaking, fitting a model is very easy in terms of the code. You can see that the output of this cell just prints the

same options we've already seen. While the fitting procedure did not return anything aside from this output, a very important change has taken place. The `my_new_lr` model object is now a trained model. We say that this change happened **in place** since no new object was created; the existing object, `my_new_lr`, has been modified. This is similar to modifying a DataFrame in place. We can now use our trained model to make predictions using the features of new samples, that the model has never "seen" before. Let's try the next 10 rows from the **EDUCATION** feature.

We can select and view these features using a new variable, `new_X`:

```
new_X = df['EDUCATION'][10:20].values.reshape(-1,1)
new_X
array([[3],
       [1],
       [2],
       [2],
       [1],
       [3],
       [1],
       [1],
       [1],
       [3]])
```

Figure 2.3: New features to make predictions for

Making predictions is done like this:

```
my_new_lr.predict(new_X)
```

Here is the output:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

We can also view the true values corresponding to these predictions, since this data is labeled:

```
df['default payment next month'][10:20].values
```

Here is the output:

```
Out[17]:array([0, 0, 0, 1, 0, 0, 1, 0, 0, 0])
```

Here, we've illustrated several things. After getting our new feature values, we've called the **.predict** method on the trained model. Notice that the only argument to this method is a set of features, that is, an "X" that we've called **new_X**.

How well did our little model do? We may naively observe that since the model predicted all 0s, and 80% of the true labels are 0s, we were right 80% of the time, which seems pretty good. On the other hand, we entirely failed to successfully predict any 1s. So, if those were important, we did not actually do very well. While this is just an example to get you familiar with how scikit-learn works, it's worth considering what a "good" prediction might look like for this problem. We will get into the details of assessing model predictive capabilities shortly. For now, congratulate yourself on having gotten your hands dirty with some real data and fitting your first machine learning model.

Generating Synthetic Data

In the following exercise, you will walk through the model fitting process on your own. We'll motivate this process using a linear regression, one of the best-known mathematical models, which should be familiar from basic statistics. It's also called a line of best fit. If you don't know what it is, you could consult a basic statistics resource, although the intent here is to illustrate the mechanics of model fitting in sci-kit learn, as opposed to understanding the model in detail. We'll work on that later in the book for other mathematical models that we'll apply to the case study, such as logistic regression. In order to have data to work with, you will generate your own **synthetic data**. Synthetic data is a valuable learning tool for exploring models, illustrating mathematical concepts, and for conducting thought experiments to test various ideas. In order to make synthetic data, we will again illustrate here how to use NumPy's **random** library to generate random numbers, as well as matplotlib's **scatter** and **plot** functions to create scatter and line plots. In the exercise, we'll use scikit-learn for the linear regression part.

To get started, we use NumPy to make a one-dimensional array of feature values, **X**, consisting of 1,000 random real numbers (in other words, not just integers but decimals as well) between 0 and 10. We again use a **seed** for the random number generator. Next, we use the **.uniform** method of **default_rng** (random number generator), which draws from the uniform distribution: it's equally likely to choose any number between **low** (inclusive) and **high** (exclusive), and will return an array of whatever **size** you specify. We create a one-dimensional array (that is, a vector) with 1,000 elements, then examine the first 10. All of this can be done using the following code:

```
from numpy.random import default_rng
rg = default_rng(12345)
X = rg.uniform(low=0.0, high=10.0, size=(1000,))
X[0:10]
```

The output should appear as follows:

```
array ([2.27336022),  3.1675834 ,  7.97365457,  6.76254671,  3.91109551,
        3.32813928 ,  5.98308754,  1.86734186,  6.72756044,  9.41802865])
```

Figure 2.4: Creating random, uniformly distributed numbers with NumPy

Data for Linear Regression

Now we need a response variable. For this example, we'll generate data that follows the assumptions of linear regression: the data will exhibit a linear trend against the feature, but have normally distributed errors:

$$y = ax + b + N(\mu, \sigma)$$

Figure 2.5: Linear equation with Gaussian noise

Here, *a* is the slope, *b* is the intercept, and the Gaussian noise has a mean of μ with a standard deviation of σ . In order to write code to implement this, we need to make a corresponding vector of responses, **y**, which are calculated as the slope times the feature array, **X**, plus some

Gaussian noise (again using NumPy), and an intercept. The noise will be an array of 1,000 data points with the same shape (**size**) as the feature array, **X**, where the mean of the noise (**loc**) is 0 and the standard deviation (**scale**) is 1. This will add a little "spread" to our linear data:

```
slope = 0.25
intercept = -1.25
y = slope * X + rg.normal(loc=0.0, scale=1.0, size=
(1000,))\
    + intercept
```

Now we'd like to visualize this data. We will use matplotlib to plot **y** against the feature **X** as a scatter plot. First, we use **.rcParams** to set the resolution (**dpi** = dots per inch) for a nice crisp image. Then we create the scatter plot with **plt.scatter**, where **X** and **y** are the first two arguments, respectively, and the **s** argument specifies a size for the dots.

This code can be used for plotting:

```
mpl.rcParams['figure.dpi'] = 400
plt.scatter(X,y,s=1)
plt.xlabel('X')
plt.ylabel('y')
```

After executing these cells, you should see something like this in your notebook:

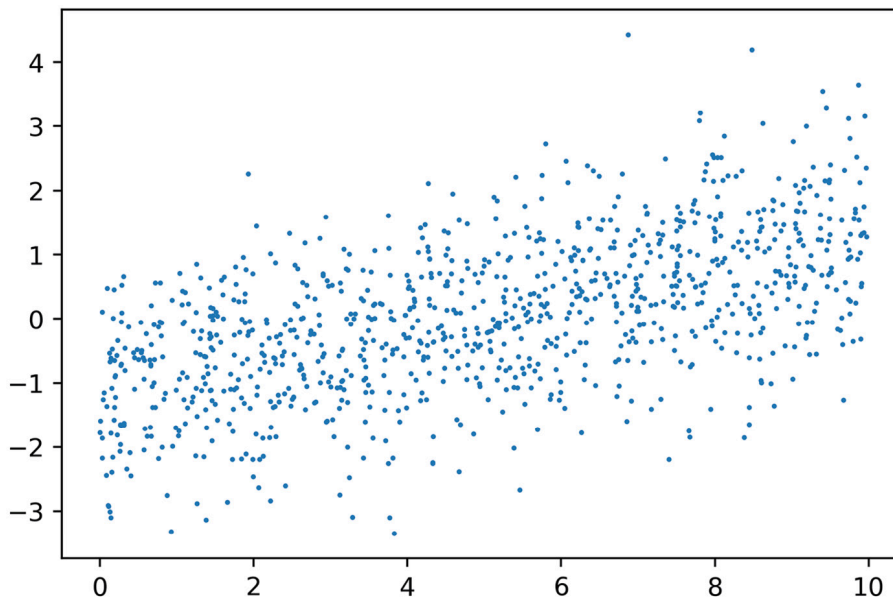


Figure 2.6: Plot the noisy linear relationship

Looks like some noisy linear data, just like we hoped. Now let's model it.

Note

If you're reading the print version of this book, you can download and browse the color versions of some of the images in this chapter by visiting the following link: <https://packt.link/0dbUp>.

Exercise 2.01: Linear Regression in Scikit-Learn

In this exercise, we will take the synthetic data we just generated and determine a line of best fit, or linear regression, using scikit-learn. The first step is to import a linear regression model class from scikit-learn and create an object from it. The import is similar to the **LogisticRegression** class we worked with previously. As with any model class, you should observe what all the default options are. Notice that for linear regression, there are not that many options to specify: you will use the defaults for this exercise. The default settings include **fit_intercept=True**, meaning the regression model will include an intercept term. This is certainly appropriate since we added an intercept to the synthetic data. Perform the following steps to com-

plete the exercise, noting that the code creating the data for linear regression from the preceding section must be run first in the same notebook (as seen on GitHub):

Note

The Jupyter notebook for this exercise can be found here:

<https://packt.link/IaoyM>.

1. Execute this code to import the linear regression model class and instantiate it with all the default options:

```
from sklearn.linear_model import LinearRegression
lin_reg = LinearRegression(fit_intercept=True,
                           normalize=False, \
                           copy_X=True, n_jobs=None)

lin_reg
```

You should see the following output:

```
Out[11]: LinearRegression()
```

No options are displayed since we used all the defaults. Now we can fit the model using our synthetic data, remembering to reshape the feature array (as we did earlier) so that that samples are along the first dimension. After fitting the linear regression model, we examine `lin_reg.intercept_`, which contains the intercept of the fitted model, as well as `lin_reg.coef_`, which contains the slope.

2. Run this code to fit the model and examine the coefficients:

```
lin_reg.fit(X.reshape(-1,1), y)
print(lin_reg.intercept_)
print(lin_reg.coef_)
```

You should see this output for the intercept and slope:

```
-1.2522197212675905
[0.25711689]
```

We again see that actually fitting a model in scikit-learn, once the data is prepared and the options for the model are decided, is a trivial process. This is because all the algorithmic work of determining the model parameters is abstracted away from the user. We

will discuss this process later, for the logistic regression model we'll use on the case study data.

What about the slope and intercept of our fitted model?

These numbers are fairly close to the slope and intercept we indicated when creating the model. However, because of the random noise, they are only approximations.

Finally, we can use the model to make predictions on feature values. Here, we do this using the same data used to fit the model: the array of features, **X**. We capture the output of this as a variable, **y_pred**. This is very similar to the example shown in *Figure 2.7*, only here we are making predictions on the same data used to fit the model (previously, we made predictions on different data) and we put the output of the **.predict** method into a variable.

3. Run this code to make predictions:

```
y_pred = lin_reg.predict(X.reshape(-1,1))
```

We can plot the predictions, **y_pred**, against feature **X** as a line plot over the scatter plot of the feature and response data, like we made in *Figure 2.6*. Here, we make the addition of **plt.plot**, which produces a line plot by default, to plot the feature and the model-predicted response values for the model training data. Notice that we follow the **X** and **y** data with '**r**' in our call to **plt.plot**. This keyword argument causes the line to be red and is part of a shorthand syntax for plot formatting.

4. This code can be used to plot the raw data, as well as the fitted model predictions on this data:

```
plt.scatter(X,y,s=1)
plt.plot(X,y_pred,'r')
plt.xlabel('X')
plt.ylabel('y')
```

After executing this cell, you should see something like this:

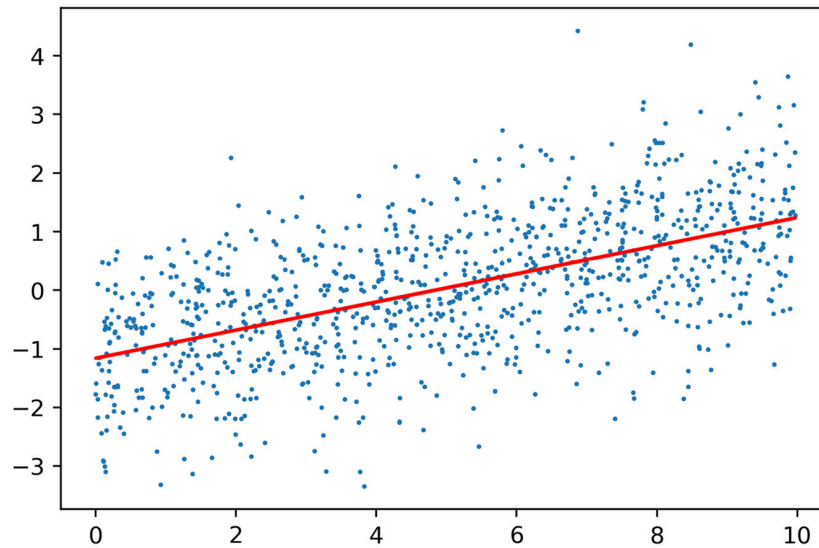


Figure 2.7: Plotting the data and the regression line

The plot looks like a line of best fit, as expected.

In this exercise, as opposed to when we called `.predict` with logistic regression, we made predictions on the same data **X** that we used to train the model. This is an important distinction. While here, we are seeing how the model "fits" the same data that it was trained on, we previously examined model predictions on new, unseen data. In machine learning, we are usually concerned with predictive capabilities: we want models that can help us know the likely outcomes of future scenarios. However, it turns out that model predictions on both the **training data** used to fit the model and the **test data**, which was not used to fit the model, are important for understanding the workings of the model. We will formalize these notions later in *Chapter 4, The Bias-Variance Trade-Off*, when we discuss the **bias-variance trade-off**.

Model Performance Metrics for Binary Classification

Before we start building predictive models in earnest, we would like to know how we can determine, once we've created a model, whether it is "good" in some sense of the word. As you may imagine, this question

has received a lot of attention from researchers and practitioners. Consequently, there is a wide variety of model performance metrics to choose from.

Note

For an idea of the range of options, have a look at the scikit-learn model evaluation page: https://scikit-learn.org/stable/modules/model_evaluation.html#model-evaluation.

When selecting a model performance metric to assess the predictive quality of a model, it's important to keep two things in mind.

Appropriateness of the metric for the problem

Metrics are typically only defined for a specific class of problems, such as classification or regression. For a binary classification problem, several metrics characterize the correctness of the yes or no question that the model answers. An additional level of detail here is how often the model is correct for each class, the positive and negative classes. We will go into detail on these metrics here. On the other hand, regression metrics are aimed at measuring how close a prediction is to the target quantity. If we are trying to predict the price of a house, how close did we come? Are we systematically over- or under-estimating? Are we getting the more expensive houses wrong but the cheaper ones right? There are many possible ways to look at regression metrics.

Does the metric answer the business question?

Whatever class of problem you are working on, there will be many choices for the metric. Which one is the right one? And even then, how do you know if a model is "good enough" in terms of the metric? At some level, this is a subjective question. However, we can be objective when we consider what the goal of the model is. In a business context, typical goals are to increase profit or reduce loss. Ultimately, you need

to unify your business question, which is often related to money in some way, and the metric you will use to judge your model.

For example, in our credit default problem, is there a particularly high cost associated with not correctly identifying accounts that will default? Is this more important than potentially misclassifying some of the accounts that won't default?

Later in the book, we'll incorporate the concept of relative costs and benefits of correct and incorrect classifications in our problem and conduct a financial analysis. First, we'll introduce you to the most common metrics used to assess the predictive quality of binary classification models, the kinds of model we need to build for our case study.

Splitting the Data: Training and Test Sets

In the scikit-learn introduction of this chapter, we introduced the concept of using a trained model to make predictions on new data that the model had never "seen" before. It turns out this is a foundational concept in predictive modeling. In our quest to create a model that has predictive capabilities, we need some kind of measure of how well the model can make predictions on data that were not used to fit the model. This is because in fitting a model, the model becomes "specialized" at learning the relationship between features and response on the specific set of labeled data that were used for fitting. While this is nice, in the end we want to be able to use the model to make accurate predictions on new, unseen data, for which we don't know the true value of the labels.

For example, in our case study, once we deliver the trained model to our client, they will then generate a new dataset of features like those we have now, except instead of spanning the period from April to September, they will span from May to October. And our client will be

using the model with these features, to predict whether accounts will default in November.

In order to know how well we can expect our model to predict which accounts will actually default in November (which won't be known until December), we can take our current dataset and reserve some of the data we have, with known labels, from the model training process. This data is referred to as **test data** and may also be called **out-of-sample data** since it consists of samples that were not used in training the model. Those samples used to train the model are called **training data**. The practice of holding out a set of test data gives us an idea of how the model will perform when it is used for its intended purpose, to make predictions on samples that were not included during model training. In this chapter, we'll create an example train/test split to illustrate different binary classification metrics.

We will use the convenient **train_test_split** functionality of scikit-learn to split the data so that 80% will be used for training, holding 20% back for testing. These percentages are a common way to make such a split; in general, you want enough training data to allow the algorithm to adequately "learn" from a representative sample of data. However, these percentages are not set in stone. If you have a very large number of samples, you may not need as large a percentage of training data, since you will be able to achieve a pretty large, representative training set with a lower percentage. We encourage you to experiment with different sizes and see the effect. Also, be aware that every problem is different with respect to how much data is needed to effectively train a model. There is no hard and fast rule for sizing your training and test sets.

For our 80/20 split, we can use the code shown in the following snippet:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split\
```

```

(df[ 'EDUCATION' ]\
 .values.reshape(-1,
1),\
 df[ 'default
payment'\
 ' next month']\
 .values,
test_size=0.2,\
 random_state=24)

```

Notice that we've set **test_size** to **0.2**, or 20%. The size of the training data will be automatically set to the remainder, 80%. Let's examine the shapes of our training and test data, to see whether they are as expected, as shown in the following output:

```

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)

(21331, 1)
(5333, 1)
(21331,)
(5333,)

```

Figure 2.8: Shape of training and test sets

You should confirm for yourself that the number of samples (rows) in the training and test sets is consistent with an 80/20 split.

In making the train/test split, we've also set the **random_state** parameter, which is a random number seed. Using this parameter allows a consistent train/test split across runs of this notebook. Otherwise, the random splitting procedure would select a different 20% of the data for testing each time the code was run.

The first argument to **train_test_split** is the features, in this case just **EDUCATION**, and the second argument is the response. There are four outputs: the features of the samples in the training and test sets, respectively, and the corresponding response variables that go with

these sets of features. All this function has done is randomly select 20% of the row indices from the dataset and subset out these features and responses as test data, leaving the rest for training. Now that we have our training and test data, it's good to make sure the nature of the data is the same between these sets. In particular, is the fraction of the positive class similar? You can observe this in the following output:

```
np.mean(y_train)
```

```
0.223102526838873
```

```
np.mean(y_test)
```

```
0.21657603600225014
```

Figure 2.9: Class fractions in training and test data

The positive class fractions in the training and test data are both about 22%. This is good, as we can say that the training set is representative of the test set. In this case, since we have a pretty large dataset with tens of thousands of samples, and the classes are not too imbalanced, we didn't have to take precautions to ensure this happens.

However, you can imagine that if the dataset were smaller, and the positive class very rare, it may be that the class fractions would be noticeably different between the training and test sets, or worse yet, there might be no positive samples at all in the test set. In order to guard against such scenarios, you could use **stratified sampling**, with the **stratify** keyword argument of **train_test_split**. This procedure also makes a random split of the data into training and test sets but guarantees that the class fractions will be equal or very similar.

Note

Out-of-time testing

If your data contains both features and responses that span a substantial period of time, it's a good practice to try making your train/test split

over time. For example, if you have two years of data with features and responses from every month, you may wish to try sequentially training the model on 12 months of data and testing on the next month, or the month after that, depending on what is operationally feasible when the model will be used. You could repeat this until you've exhausted your data, to get a few different test scores. This will give you useful insights into model performance because it simulates the actual conditions the model will face when it is deployed: a model trained on old features and responses will be used to make predictions on new data. In the case study, the responses only come from one point in time (credit defaults within one month), so this is not an option here.

Classification Accuracy

Now we proceed to fit an example model to illustrate binary classification metrics. We will continue to use logistic regression with near-default options, choosing the same options we demonstrated in *Chapter 1, Data Exploration and Cleaning*:

```
from sklearn.linear_model import LogisticRegression

example_lr = LogisticRegression(C=0.1, class_weight=None,
                                dual=False, fit_intercept=True,
                                intercept_scaling=1, max_iter=100,
                                multi_class='auto', n_jobs=None,
                                penalty='l2', random_state=None,
                                solver='liblinear', tol=0.0001,
                                verbose=0, warm_start=False)
```

Figure 2.10: Loading the model class and creating a model object

Now we proceed to train the model, as you might imagine, using the labeled data from our training set. We proceed immediately to use the trained model to make predictions on the features of the samples from the held-out test set:

```
example_lr.fit(X_train, y_train)

LogisticRegression(C=0.1, solver='liblinear')

y_pred = example_lr.predict(X_test)
```

Accuracy

Figure 2.11: Training a model and making predictions on the test set

We've stored the model-predicted labels of the test set in a variable called **y_pred**. How should we now assess the quality of these predictions? We have the true labels, in the **y_test** variable. First, we will compute what is probably the simplest of all binary classification metrics: **accuracy**. Accuracy is defined as the proportion of samples that were correctly classified.

One way to calculate accuracy is to create a logical mask that is **True** whenever the predicted label is equal to the actual label, and **False** otherwise. We can then take the average of this mask, which will interpret **True** as 1 and **False** as 0, giving us the proportion of correct classifications:

```
is_correct = y_pred == y_test

np.mean(is_correct)

0.7834239639977498
```

Figure 2.12: Calculating classification accuracy with a logical mask

This indicates that the model is correct 78% of the time. While this is a pretty straightforward calculation, there are actually easier ways to calculate accuracy using the convenience of scikit-learn. One way is to use the trained model's **.score** method, passing the features of the test data to make predictions on, as well as the test labels. This method makes the predictions and then does the same calculation we performed previously, all in one step. Or, we could import scikit-learn's **metrics** library, which includes many model performance metrics,

such as **accuracy_score**. For this, we pass the true labels and the predicted labels:

```
example_lr.score(X_test, y_test)

0.7834239639977498

from sklearn import metrics

metrics.accuracy_score(y_test, y_pred)

0.7834239639977498
```

Figure 2.13: Calculating classification accuracy with scikit-learn

These all give the same result, as they should. Now that we know how accurate the model is, how do we interpret this metric? On the surface, an accuracy of 78% may sound good. We are getting most of the predictions right. However, an important test for the accuracy of binary classification is to compare things to a very simple hypothetical model that only makes one prediction: this hypothetical model predicts the majority class for every sample, no matter what the features are. While in practice this model is useless, it provides an important extreme case with which to compare the accuracy of our trained model. Such extreme cases are sometimes referred to as null models.

Think about what the accuracy of such a null model would be. In our dataset, we know that about 22% of the samples are positive. So, the negative class is the majority class, with the remaining 78% of the samples. Therefore, a null model for this dataset, which always predicts the majority negative class, will be right 78% of the time. Now when we compare our trained model here to such a null model, it becomes clear that an accuracy of 78% is actually not very useful. We can get the same accuracy with a model that doesn't pay any attention to the features.

While we can interpret accuracy in terms of a majority-class null model, there are other binary classification metrics that delve a little

deeper into how the model is performing for negative, as well as positive samples separately.

True Positive Rate, False Positive Rate, and Confusion Matrix

In binary classification, there are just two labels to consider: positive and negative. As a more descriptive way to look at model performance than the accuracy of prediction across all samples, we can also look at the accuracy of only those samples that have a positive label. The proportion of these that we successfully predict as positive is called the **true positive rate (TPR)**. If we say that **P** is the number of samples in the **positive class** in the test data, and **TP** is the number of **true positives**, defined as the number of positive samples that were predicted to be positive by the model, then the TPR is as follows:

$$TPR = \frac{TP}{P}$$

Figure 2.14: TPR equation

The flip side of the true positive rate is the **false negative rate (FNR)**. This is the proportion of positive test samples that we incorrectly predicted as negative. Such errors are called **false negatives (FN)** and the **false negative rate (FNR)** is calculated as follows:

$$FNR = \frac{FN}{P}$$

Figure 2.15: FNR equation

Since all the positive samples are either correctly or incorrectly predicted, the sum of the number of true positives and the number of false negatives equals the total number of positive samples. Mathematically, $P = TP + FN$, and therefore, using the definitions of TPR and FNR, we have the following:

$$TPR + FNR = 1$$

Figure 2.16: The relation between the TPR and FNR

Since the TPR and FNR sum to 1, it's sufficient to just calculate one of them.

Similar to the TPR and FNR, there is the **true negative rate (TNR)** and the **false positive rate (FPR)**. If N is the number of **negative** samples, the sum of **true negative** samples (TN) is the number of these that are correctly predicted, and the sum of **false positive (FP)** samples is the number incorrectly predicted as positive:

$$TNR = \frac{TN}{N}$$

Figure 2.17: TNR equation

$$FPR = \frac{FP}{N}$$

Figure 2.18: FPR equation

$$TNR + FPR = 1$$

Figure 2.19: Relation between the TNR and FPR

True and false positives and negatives can be conveniently summarized in a table called a **confusion matrix**. A confusion matrix for a binary classification problem is a 2 x 2 matrix where the true class is along one axis and the predicted class is along the other. The confusion matrix gives a quick summary of how many true and false positives and negatives there are:

		Predicted class	
		N	P
True class	N	TN	FP
	P	FN	TP

Figure 2.20: The confusion matrix for binary classification

Since we hope to make correct classifications, we hope that the **diagonal** entries (that is, the entries along a diagonal line from the top left to the bottom right: TN and TP) of the confusion matrix are relatively large, while the off-diagonals are relatively small, as these represent

incorrect classifications. The accuracy metric can be calculated from the confusion matrix by adding up the entries on the diagonal, which are predictions that are correct, and dividing by the total number of all predictions.

Exercise 2.02: Calculating the True and False Positive and Negative Rates and Confusion Matrix in Python

In this exercise, we'll use the test data and model predictions from the logistic regression model we created previously, using only the **EDUCATION** feature. We will illustrate how to manually calculate the true and false positive and negative rates, as well as the numbers of true and false positives and negatives needed for the confusion matrix. Then we will show a quick way to calculate a confusion matrix with scikit-learn. Perform the following steps to complete the exercise, noting that some code from the previous section must be run before doing this exercise (as seen on GitHub):

Note

The Jupyter notebook for this exercise can be found here:

<https://packt.link/S02kz>.

1. Run this code to calculate the number of positive samples:

```
P = sum(y_test)
P
```

The output should appear like this:

```
1155
```

Now we need the number of true positives. These are samples where the true label is 1 and the prediction is also 1. We can identify these with a logical mask for the samples that are positive (**y_test==1**) **AND** (& is the logical **AND** operator in Python) have a positive prediction (**y_pred==1**).

2. Use this code to calculate the number of true positives:

```
TP = sum( (y_test==1) & (y_pred==1) )
```

```
TP
```

Here is the output:

```
0
```

The true positive rate is the proportion of true positives to positives, which of course would be 0 here.

3. Run the following code to obtain the TPR:

```
TPR = TP/P  
TPR
```

You will obtain the following output:

```
0.0
```

Similarly, we can identify the false negatives.

4. Calculate the number of false negatives with this code:

```
FN = sum( (y_test==1) & (y_pred==0) )  
FN
```

This should output the following: 1155

We'd also like the FNR.

5. Calculate the FNR with this code:

```
FNR = FN/P  
FNR
```

This should output the following:

```
1.0
```

What have we learned from the true positive and false negative rates?

First, we can confirm that they sum to 1. This fact is easy to see because the $TPR = 0$ and the $FPR = 1$. What does this tell us about our model? On the test set, at least for the positive samples, the model has in fact acted as a majority-class null model. Every positive sample was predicted to be negative, so none of them was correctly predicted.

6. Let's find the TNR and FPR of our test data. Since these calculations are very similar to those we looked at previously, we show them all at once and illustrate a new Python function:

```

N = sum(y_test==0)
N
4178

TN = sum( (y_test==0) & (y_pred==0))
TN
4178

FP = sum( (y_test==0) & (y_pred==1))
FP
0

TNR = TN/N
FPR = FP/N
print('The true negative rate is {} and the false positive rate is {}'.format(TNR, FPR))
The true negative rate is 1.0 and the false positive rate is 0.0

```

Figure 2.21: Calculating true negative and false positive rates and printing them

In addition to calculating the TNR and FPR in a similar way that we had previously with the TPR and FNR, we demonstrate the **print** function in Python along with the **.format** method for strings, which allows substitution of variables in locations marked by curly braces {}. There is a range of options for formatting numbers, such as including a certain number of decimal places.

Note

For additional details, refer to

<https://docs.python.org/3/tutorial/inputoutput.html>.

Now, what have we learned here? In fact, our model behaves exactly like the majority-class null model for all samples, both positive and negative. It's clear we're going to need a better model.

While we have manually calculated all the entries of the confusion matrix in this exercise, in scikit-learn there is a quick way to do this. Note that in scikit-learn, the true class is along the vertical axis and the predicted class is along the horizontal axis of the confusion matrix, as we presented earlier.

7. Create a confusion matrix in scikit-learn with this code:

```
metrics.confusion_matrix(y_test, y_pred)
```

You will obtain the following output:

```
array([[4178,    0],
       [1155,    0]])
```

Figure 2.22: The confusion matrix for our example model

All the information we need to calculate the TPR, FNR, TNR, and FPR is contained in the confusion matrix. We also note that there are many more classification metrics that can be derived from the confusion matrix. In fact, some of these are actually synonyms for ones we've already examined here. For example, the TPR is also called **recall** and **sensitivity**. Along with recall, another metric that is often used for binary classification is **precision**: this is the proportion of positive predictions that are correct (as opposed to the proportion of positive samples that are correctly predicted). We'll get more experience with precision in the activity for this chapter.

Note

Multiclass classification

Our case study involves a binary classification problem, with only two possible outcomes: the account does or does not default. Another important type of machine learning classification problem is multiclass classification. In multiclass classification, there are several possible mutually exclusive outcomes. A classic example is image recognition of handwritten digits; a handwritten digit should be only one of 0, 1, 2, ... 9.

Although multiclass classification is outside the scope of this book, the metrics we are learning now for binary classification can be extended to the multiclass setting.

Discovering Predicted Probabilities: How Does Logistic Regression Make Predictions?

Now that we're familiar with accuracy, true and false positives and negatives, and the confusion matrix, we can explore new ways of using logistic regression to learn about more advanced binary classification metrics. So far, we've only considered logistic regression as a "black box" that can learn from labeled training data and then make binary predictions on new features. While we will learn about the

workings of logistic regression in detail later in the book, we can begin to peek inside the black box now.

One thing to understand about how logistic regression works is that the raw predictions – in other words, the direct outputs from the mathematical equation that defines logistic regression – are not binary labels. They are actually **probabilities** on a scale from 0 to 1 (although, technically, the equation never allows the probabilities to be exactly equal to 0 or 1, as we'll see later). These probabilities are only transformed into binary predictions through the use of a **threshold**. The threshold is the probability above which a prediction is declared to be positive, and below which it is negative. The threshold in scikit-learn is 0.5. This means any sample with a predicted probability of at least 0.5 is identified as positive, and any with a predicted probability < 0.5 is decided to be negative. However, we are free to use any threshold we want. In fact, choosing the threshold is one of the key flexibilities of logistic regression, as well as other machine learning classification algorithms that estimate probabilities of class membership.

Exercise 2.03: Obtaining Predicted Probabilities from a Trained Logistic Regression Model

In the following exercise, we will get familiar with the predicted probabilities of logistic regression and how to obtain them from a scikit-learn model.

We can begin to discover predicted probabilities by further examining the methods available to us on the logistic regression model object that we trained earlier in this chapter. Recall that before, once we trained the model, we could then make binary predictions using the values of features from new samples by passing these values to the **.predict** method of the trained model. These are predictions made on the assumption of a threshold of 0.5.

However, we can directly access the predicted probabilities of these samples, using the `.predict_proba` method. Perform the following steps to complete the exercise, keeping in mind that you will need to recreate the same model trained previously in the chapter if you are starting a new notebook:

Note

The Jupyter notebook for this exercise can be found here:

<https://packt.link/yDyQn>. The notebook contains the prerequisite steps of training the model and should be executed prior to the first step shown here.

1. Obtain the predicted probabilities for the test samples using this code:

```
y_pred_proba = example_lr.predict_proba(X_test)
y_pred_proba
```

The output should be as follows:

```
array([[0.77423402, 0.22576598],
       [0.77423402, 0.22576598],
       [0.78792915, 0.21207085],
       ...,
       [0.78792915, 0.21207085],
       [0.78792915, 0.21207085],
       [0.78792915, 0.21207085]])
```

Figure 2.23: Predicted probabilities of the test data

We see in the output of this, which we've stored in `y_pred_proba`, that there are two columns. This is because there are two classes in our classification problem: negative and positive. Assuming the negative labels are coded as 0 and the positives as 1, as they are in our data, scikit-learn will report the probability of negative class membership as the first column, and positive class membership as the second.

Since the two classes are mutually exclusive and are the only options, the sum of predicted probabilities for the two classes should equal 1 for every sample. Let's confirm this.

First, we can use `np.sum` over the first dimension (columns) to calculate the sum of probabilities for each sample.

2. Calculate the sum of predicted probabilities for each sample with this code:

```
prob_sum = np.sum(y_pred_proba,1)
prob_sum
```

The output is as follows:

```
array([1., 1., 1., ..., 1., 1., 1.])
```

It certainly looks like all 1s. We should check to see that the result is the same shape as the array of test data labels.

3. Check the array shape with this code:

```
prob_sum.shape
```

This should output the following:

```
(5333,)
```

Good; this is the expected shape. Now, to check that each value is 1.

We use **np.unique** to show all the unique elements of this array.

This is similar to **DISTINCT** in SQL. If all the probability sums are indeed 1, there should only be one unique element of the probability array: 1.

4. Show all unique array elements with this code:

```
np.unique(prob_sum)
```

This should output the following:

```
array([1.])
```

After confirming our belief in the predicted probabilities, we note that since class probabilities sum to 1, it's sufficient to just consider the second column, the predicted probability of positive class membership. Let's capture these in an array.

5. Run this code to put the second column of the predicted probabilities array (predicted probability of membership in the positive class) in an array:

```
pos_proba = y_pred_proba[:,1]
pos_proba
```

The output should be as follows:

```
array([0.22576598, 0.22576598, 0.21207085, ..., 0.21207085, 0.21207085,
       0.21207085])
```

Figure 2.24: Predicted probabilities of positive class membership

What do these probabilities look like? One way to find out, and a good diagnostic for model output, is to plot the predicted probabilities. A histogram is a natural way to do this, for which we can use the matplotlib function, **hist()**. Note that if you execute a cell with only the histogram function, you will get the output of the NumPy histogram function returned before the plot. This includes the number of samples in each bin and the locations of the bin edges.

6. Execute this code to see histogram output and an unformatted plot (not shown here):

```
plt.hist(pos_proba)
```

The output is as follows:

```
(array([1883.,    0.,    0., 2519.,    0.,    0.,  849.,    0.,    0.,
        82.]),
 array([0.21207085, 0.21636321, 0.22065556, 0.22494792, 0.22924027,
        0.23353263, 0.23782498, 0.24211734, 0.24640969, 0.25070205,
        0.2549944 ]),
 <BarContainer object of 10 artists>)
```

Figure 2.25: Details of histogram calculation

This may be useful information for you and could also be obtained directly from the **np.histogram()** function. However, here we're mainly interested in the plot, so we adjust the font size and add some axis labels.

7. Run this code for a formatted histogram plot of predicted probabilities:

```
mpl.rcParams['font.size'] = 12
plt.hist(pos_proba)
plt.xlabel('Predicted probability of positive class
'\
          'for test data')
plt.ylabel('Number of samples')
```

The plot should look like this:

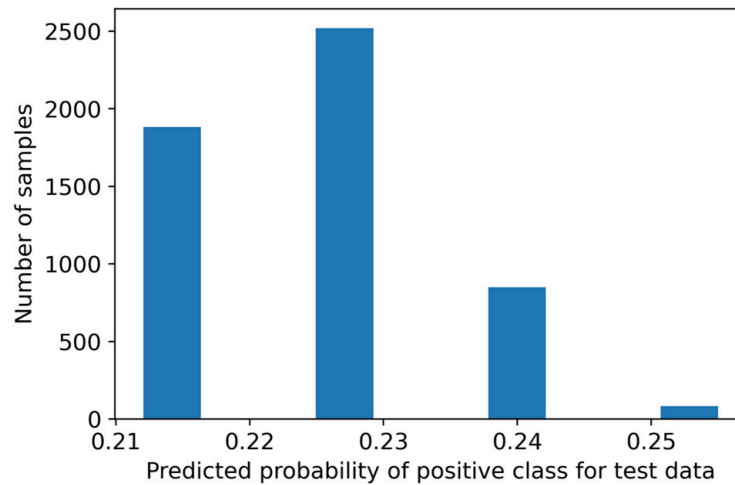


Figure 2.26: Histogram plot of predicted probabilities

Notice that in the histogram of probabilities, there are only four bins that actually have samples in them, and they are spaced fairly far apart. This is because there are only four unique values for the **EDUCATION** feature, which is the only feature in our example model. Also, notice that all the predicted probabilities are below 0.5. This is the reason every sample was predicted to be negative, using the 0.5 threshold. We can imagine that if we set our threshold below 0.5, we would get different results. For example, if we set the threshold at 0.25, all of the samples in the smallest bin to the far right of *Figure 2.26* would be classified as positive, since the predicted probability for all of these is above 0.25. It would be informative for us if we could see how many of these samples actually had positive labels. Then we could see whether moving our threshold down to 0.25 would improve the performance of our classifier by classifying the samples in the rightmost bin as positive.

In fact, we can visualize this easily, using a **stacked histogram**.

This will look a lot like the histogram in *Figure 2.27*, except that the negative and positive samples will be colored differently. First, we need to distinguish between positive and negative samples in the predicted probabilities. We can do this by indexing our array of predicted probabilities with logical masks; first to get positive samples, where `y_test == 1`, and then to get negative samples, where `y_test == 0`.

8. Isolate the predicted probabilities for positive and negative samples with this code:

```
pos_sample_pos_proba = pos_proba[y_test==1]
neg_sample_pos_proba = pos_proba[y_test==0]
```

Now we want to plot these as a stacked histogram. The code is similar to the histogram we already created, except that we will pass a list of arrays to be plotted, which are the arrays of probabilities for positive and negative samples we just created, and a keyword indicating we'd like the bars to be stacked, as opposed to plotted side by side. We'll also create a legend so that the colors are clearly identifiable on the plot.

9. Plot a stacked histogram using this code:

```
plt.hist([pos_sample_pos_proba,
neg_sample_pos_proba],\
         histtype='barstacked')
plt.legend(['Positive samples', 'Negative samples'])
plt.xlabel('Predicted probability of positive class')
plt.ylabel('Number of samples')
```

The plot should look like this:

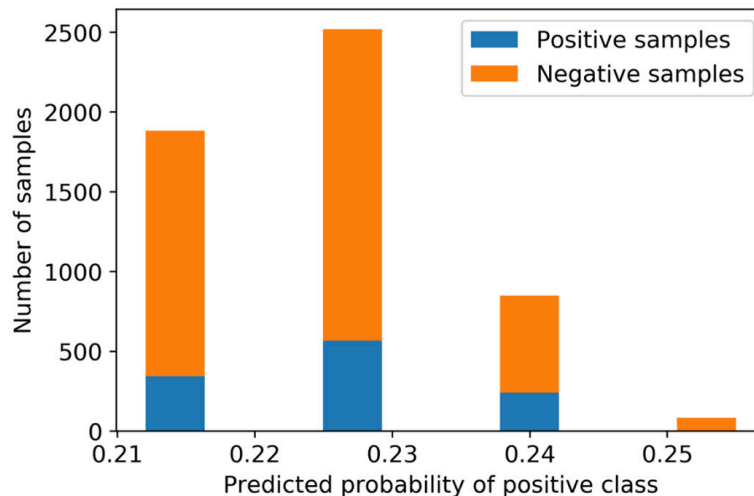


Figure 2.27: Stacked histogram of predicted probabilities by class

The plot shows us the true labels of the samples for each predicted probability. Now we can consider what the effect would be of lowering the threshold to 0.25. Take a moment and think about what this would

mean, keeping in mind that any sample with a predicted probability at or above the threshold would be classified as positive.

Since nearly all the samples in the small bin to the right of *Figure 2.28* are negative samples, if we were to decrease the threshold to 0.25, we would erroneously classify these as positive samples and increase our FPR. At the same time, we still wouldn't have managed to classify many, if any, positive samples correctly, so our TPR wouldn't increase very much at all. Making this change would appear to decrease the accuracy of the model.

The Receiver Operating Characteristic (ROC) Curve

Deciding on a threshold for a classifier is a question of finding the "sweet spot" where we are successfully recovering enough true positives, without incurring too many false positives. As the threshold is lowered more and more, there will be more of both. A good classifier will be able to capture more true positives without the expense of a large number of false positives. What would be the effect of lowering the threshold even more, with the predicted probabilities from the previous exercise? It turns out there is a classic method of visualization in machine learning, with a corresponding metric that can help answer this kind of question.

The **receiver operating characteristic (ROC)** curve is a plot of the pairs of TPRs (*y-axis*) and FPRs (*x-axis*) that result from lowering the threshold down from 1 all the way to 0. You can imagine that if the threshold is 1, there are no positive predictions since a logistic regression only predicts probabilities strictly between 0 and 1 (endpoints not included). Since there are no positive predictions, the TPR and the FPR are both 0, so the ROC curve starts out at (0, 0). As the threshold is lowered, the TPR will start to increase, hopefully faster than the FPR if it's a good classifier. Eventually, when the threshold is lowered all the way to 0, every sample is predicted to be positive, including all the samples that are, in fact, positive, but also all the samples that are actually neg-

ative. This means the TPR is 1 but the FPR is also 1. In between these two extremes are the reasonable options for where you may want to set the threshold, depending on the relative costs and benefits of true and false positives and negatives for the specific problem being considered. In this way, it is possible to get a complete picture of the performance of the classifier at all different thresholds to decide which one to use.

We could write the code to determine the TPRs and FPRs of the ROC curve by using the predicted probabilities and varying the threshold from 1 to 0. Instead, we will use scikit-learn's convenient functionality, which will take the true labels and predicted probabilities as inputs and return arrays of TPRs, FPRs, and the thresholds that lead to them. We will then plot the TPRs against the FPRs to show the ROC curve. Run this code to use scikit-learn to generate the arrays of TPRs and FPRs for the ROC curve, importing the **metrics** module if needed:

```
from sklearn import metrics
fpr, tpr, thresholds = metrics.roc_curve(y_test,
pos_proba)
```

Now we need to produce a plot. We'll use **plt.plot**, which will make a line plot using the first argument as the x values (FPRs), the second argument as the y values (TPRs), and the shorthand **'*-'** to indicate a line plot with star symbols where the data points are located. We add a straight-line plot from (0, 0) to (1, 1), which will appear in red (**'r'**) and as a dashed line (**'--'**). We've also given the plot a legend (which we'll explain shortly), as well as axis labels and a title. This code produces the ROC plot:

```
plt.plot(fpr, tpr, '*-')
plt.plot([0, 1], [0, 1], 'r--')
plt.legend(['Logistic regression', 'Random chance'])
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.title('ROC curve')
```

And the plot should look like this:

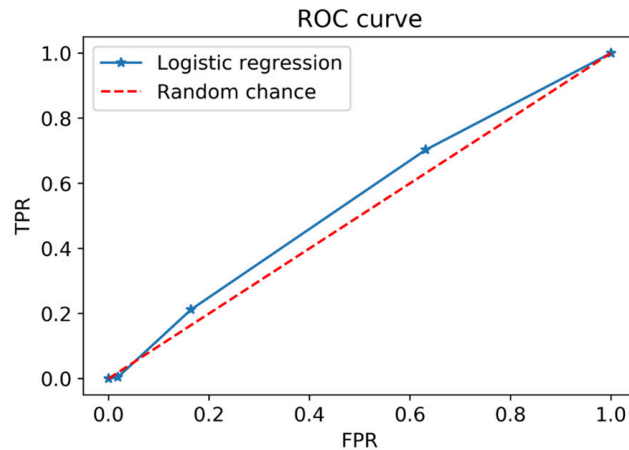


Figure 2.28: ROC curve for our logistic regression, with a line of random chance shown for comparison

What have we learned from our ROC curve? We can see that it starts at (0,0) with a threshold high enough so that there are no positive classifications. Then the first thing that happens, as we imagined previously when lowering the threshold to about 0.25, is that we get an increase in the FPR, but very little increase in the TPR. The effects of continuing to lower the threshold so that the other bars from our stacked histogram plot in *Figure 2.28* would be included as positive classifications are shown by the subsequent points on the line. We can see the thresholds that lead to these rates by examining the threshold array, which is not part of the plot. View the thresholds used to calculate the ROC curve using this code:

```
thresholds
```

The output should be as follows:

```
array([1.2549944 , 0.2549944 , 0.24007604, 0.22576598,
       0.21207085])
```

Notice that the first threshold is actually above 1; practically speaking, it just needs to be a threshold that's high enough that there are no positive classifications.

Now consider what a "good" ROC curve would look like. As we lower the threshold, we want to see the TPR increase, which means our classifier is doing a good job of correctly identifying positive samples. At the same time, ideally the FPR should not increase that much. The ROC curve of an effective classifier would hug the upper left corner of the plot: high TPR, low FPR. You can imagine that a perfect classifier would get a TPR of 1 (recovers all the positive samples) and an FPR of 0 and appear as a sort of square starting at (0,0), going up to (0,1), and finishing at (1,1). While in practice this kind of performance is highly unlikely, it gives us a limiting case.

Further consider what the **area under the curve (AUC)** of such a classifier would be, remembering integrals from calculus if you have studied it. The AUC of a perfect classifier would be 1, because the shape of the curve would be a square on the unit interval [0, 1].

On the other hand, the line labeled as "Random chance" in our plot is the ROC curve that theoretically results from flipping an unbiased coin as a classifier: it's just as likely to get a true positive as a false positive, so lowering the threshold introduces more of each in equal proportion and the TPR and FPR increase at the same rate. The AUC under this ROC would be half of the perfect classifier's, as you can see graphically, and would be 0.5.

So, in general, the ROC AUC is going to be between 0.5 and 1 (although values below 0.5 are technically possible). Values close to 0.5 indicate the model can do little better than random chance (coin flip) as a classifier, while values closer to 1 indicate better performance. The **ROC AUC** is a key metric for the quality of a classifier and is widely used in machine learning. The ROC AUC may also be referred to as the **C-statistic** (concordance statistic).

Being such an important metric, scikit-learn has a convenient way to calculate the ROC AUC. Let's see what the ROC AUC of the logistic regression classifier is, where we can pass the same information that we

did to the **roc_curve** function. Calculate the area under the ROC curve with this code:

```
metrics.roc_auc_score(y_test, pos_proba)
```

And observe the output:

```
0.5434650477972642
```

The ROC AUC for the logistic regression is pretty close to 0.5, meaning it's not a very effective classifier. This may not be surprising, considering we have expended no effort to determine which features out of the candidate pool are actually useful at this point. We're just getting used to model fitting syntax and learning the way to calculate model quality metrics using a simple model containing only the **EDUCATION** feature. Later on, by considering other features, hopefully we'll get a higher ROC AUC.

Note

ROC curve: How did it get that name?

During World War II, radar receiver operators were evaluated on their ability to judge whether something that appeared on their radar screen was in fact an enemy aircraft or not. These decisions involved the same concepts of true and false positives and negatives that we are interested in for binary classification. The ROC curve was devised as a way to measure the effectiveness of operators of radar receiver equipment.

Precision

Before embarking on the activity, we will consider the classification metric briefly introduced previously: **precision**. Like the ROC curve, this diagnostic is useful over a range of thresholds. Precision is defined as follows:

$$precision = \frac{TP}{TP + FP}$$

Figure 2.29: Precision equation

Consider the interpretation of this, in the sense of varying the threshold across the range of predicted probabilities, as we did for the ROC curve. At a high threshold, there will be relatively few samples predicted as positive. As we lower the threshold, more and more will be predicted as positive. Our hope is that as we do this, the number of true positives increases more quickly than the number of false positives, as we saw on the ROC curve. Precision looks at the ratio of the number of true positives to the sum of true and false positives. Think about the denominator here: what is the sum of true and false positives?

This sum is in fact the total number of positive predictions, since all positive predictions will be either correct or incorrect. So, precision measures the ratio of positive predictions that are correct to all positive predictions. For this reason, it is also called the **positive predictive value**. If there are very few positive samples, precision gives a more critical assessment of the quality of a classifier than the ROC AUC. As with the ROC curve, there is a convenient function in scikit-learn to calculate precision, together with recall (also known as the TPR), over a range of thresholds: `metrics.precision_recall_curve`. Precision and recall are often plotted together to assess the quality of positive predictions as far as what fraction are correct, while at the same time considering what fraction of the positive class a model is able to identify. We'll plot a precision-recall curve in the following activity.

Why might precision be a useful measure of classifier performance? Imagine that for every positive model prediction, you are going to take some expensive course of action, such as a time-consuming review of content that was flagged as inappropriate by an automated procedure. False positives would waste the valuable time of human reviewers. You would want to be sure that you were making the right decisions

on what content received a detailed review. Precision could be a good metric to use in this situation.

Activity 2.01: Performing Logistic Regression with a New Feature and Creating a Precision-Recall Curve

In this activity, you'll train a logistic regression model using a feature besides **EDUCATION**. Then you will graphically assess the trade-off between precision and recall, as well as calculate the area underneath a precision-recall curve. You will also calculate the ROC AUC on both the training and test sets and compare them.

Perform the following steps to complete the activity:

Note

The code and the resulting output for this activity have been loaded in a Jupyter notebook that can be found here: <https://packt.link/SvAOD>.

1. Use scikit-learn's **train_test_split** to make a new set of training and test data. This time, instead of **EDUCATION**, use **LIMIT_BAL**, the account's credit limit, as the feature.
2. Train a logistic regression model using the training data from your split.
3. Create the array of predicted probabilities for the test data.
4. Calculate the ROC AUC using the predicted probabilities and the true labels of the test data. Compare this to the ROC AUC from using the **EDUCATION** feature.
5. Plot the ROC curve.
6. Calculate the data for the **precision-recall curve** on the test data using scikit-learn's functionality.
7. Plot the precision-recall curve using matplotlib.
8. Use scikit-learn to calculate the area under the precision-recall curve. You should get a value of approximately 0.315.

9. Now recalculate the ROC AUC, except this time do it for the training data. How is this different, conceptually and quantitatively, from your earlier calculation?

Note

The Jupyter notebook containing the Python code solution for this activity can be found here: <https://packt.link/SvAOD>. Detailed step-wise solution to this activity can be found via [this link](#).

Summary

In this chapter, we finished the initial exploration of the case study data by examining the response variable. Once we became confident in the completeness and correctness of the dataset, we were prepared to explore the relation between features and response and build models.

We spent much of this chapter getting used to model fitting in scikit-learn at the technical, coding level, and learning about metrics we could use with the binary classification problem of the case study. When trying different feature sets and different kinds of models, you will need some way to tell if one approach is working better than another. Consequently, you'll need to use model performance metrics like those we learned in this chapter.

While accuracy is a familiar and intuitive metric as the percentage of correct classifications, we learned why it may not give a useful assessment of the performance of a classifier. We learned how to use a majority-class null model to tell whether an accuracy rate is truly good, or no better than what would result from simply predicting the most common class for all samples. When the data is imbalanced, accuracy is usually not the best way to judge a classifier.

In order to have a more nuanced view of how a model is performing, it's necessary to separate the positive and negative classes and assess

the accuracy of them independently. From the resulting counts of true and false positive and negative classifications, which can be summarized in a confusion matrix, we can derive several other metrics: true and false positive and negative rates. Combining true and false positives and negatives with the concept of predicted probabilities and a variable threshold of prediction, we can further characterize the usefulness of a classifier using the ROC curve, the precision-recall curve, and the areas under these curves.

With these tools, you are well equipped to answer general questions about the performance of a binary classifier in any domain you may be working in. Later in the book, we will learn about application-specific ways to assess model performance by attaching costs and benefits to true and false positives and negatives. Before that, starting in the next chapter, we will begin learning the details behind what is possibly the most popular and simplest classification model: **logistic regression**.